

Consignes :

- répondez à chaque problème **sur des feuilles séparées** et utilisez uniquement les feuilles de réponses qui vous sont données ;
 - indiquez votre nom, prénom et section sur chaque feuille de réponses ;
 - les notes de cours et personnelles **ne sont pas autorisées** ;
 - si ce n'est pas explicitement demandé dans l'énoncé :
 - **vous ne devez pas tester que les préconditions sont respectées** ;
 - **vos fonctions ne doivent pas avoir d'effet de bord** : les objets mutables passés en paramètres ne doivent pas être modifiés et rien ne doit être affiché à l'écran.
-

Problème 1 : délimiteurs

Le but de cet exercice est d'implémenter une fonction permettant de parcourir une chaîne de caractères et d'afficher tous les éléments se trouvant entre des délimiteurs successifs. Par exemple, prenons la chaîne de caractères suivante « -chat-chien-vache-cochon- » et le caractère « - » comme délimiteur ; une telle fonction affichera :

```
chat
chien
vache
cochon
```

Question :

Donnez le code python d'une fonction `tokenizer(chaine, delimiteur)`. Le paramètre `chaine` représente la chaîne de caractères à parcourir. Le paramètre `delimiteur` représente quant à lui le délimiteur à utiliser. Cette fonction vous affichera les éléments compris entre **deux** délimiteurs successifs. L'usage de la fonction `split()` est **interdit**.

Veuillez envisager les cas particuliers suivants :

- La fonction doit pas afficher de lignes vides si un ou plusieurs délimiteurs se suivent.
- Pensez au cas où la chaîne de caractères ne se termine pas ou ne commence pas par un délimiteur. Que devrait retourner cette fonction sur les chaînes « -chat-chien » ou « chat-chien- » avec « - » comme délimiteur ?

Problème 2 : ordonnancement de tâches

Un ordonnanceur de tâches a pour objectif de trouver un ordre dans lequel différentes tâches numérotées peuvent être effectuées tout en respectant certaines contraintes de précédence. Une contrainte de précédence (i, j) stipule que la tâche numéro j doit être réalisée *avant* d'effectuer la tâche numéro i .

A titre d'exemple, considérez un système informatique comprenant 4 tâches devant être réalisées, numérotées de 0 à 3. Les contraintes de précédence de ce système sont les suivantes : $(2, 0)$, $(2, 1)$, $(0, 1)$, $(3, 2)$. Un ordonnanceur de tâches trouvera l'ordre de réalisation des tâches suivant : 1, 0, 2, 3.

Un système de tâches à ordonnancer peut être encodé sous la forme d'une matrice carrée de taille $n \times n$ où n est le nombre de tâches à ordonnancer. Dans cette matrice, l'élément en position (i, j) a la valeur *True* si une contrainte de précédence (i, j) a été spécifiée, *False* sinon. Ainsi, dans l'exemple donné ci-dessus, le système serait encodé sous la forme de la matrice 4×4 suivante :

$$\begin{pmatrix} False & True & False & False \\ False & False & False & False \\ True & True & False & False \\ False & False & True & False \end{pmatrix}$$

Il n'est pas toujours possible de trouver un ordre de réalisation des tâches. Par exemple, dans un système à deux tâches, si les contraintes de précédence sont $(0, 1)$, $(1, 0)$, il n'existe aucun ordre permettant de réaliser les tâches tout en respectant les contraintes.

Questions

Donnez le code Python des fonctions suivantes :

- Une fonction `creer_matrice(n_taches, contraintes)` qui, étant donnés un nombre `n_taches` de tâches et une liste `contraintes` de couples représentant la liste des contraintes d'un système, retourne la matrice du système correspondante. Cette fonction a pour précondition que `n_taches` ≥ 1 .
- Une fonction `ordonnanceur(matrice)` qui, étant donnée une matrice encodant le système à ordonnancer, retourne une liste d'entiers représentant un ordre dans lequel les tâches peuvent être réalisées. Dans notre exemple, cette fonction retournerait `[1, 0, 2, 3]`. Cette fonction a pour précondition que la matrice fournie en paramètre encode un système pour lequel il existe toujours un ordre de réalisation des tâches. Notez que la fonction `ordonnanceur` ne doit pas avoir d'effet de bord.

Problème 3 : nombres pairs

Donnez le code Python des deux fonctions suivantes, en respectant scrupuleusement les contraintes données après leurs énoncés :

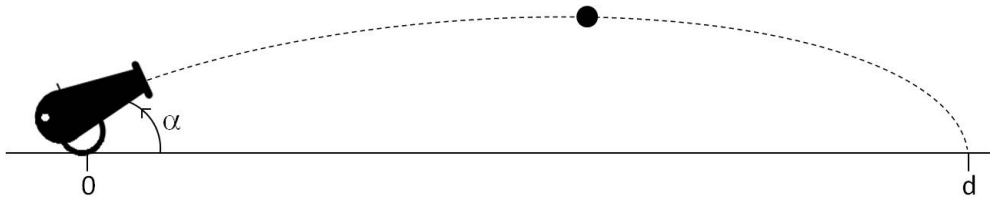
- une fonction `somme_pair(t)` qui, étant donné une liste `t` contenant des nombres naturels strictement positifs (précondition), retourne la somme des nombres pairs contenus dans `t`. Par exemple, `somme_pair([5, 2, 4, 1])` retournera `6`;
- une fonction `filtre_pair(t)` qui, étant donné une liste `t` contenant des nombres naturels strictement positifs (précondition), retourne une nouvelle liste qui ne contient les nombres pairs présents dans `t`, en respectant l'ordre original. Par exemple, `filtre_pair([5, 2, 4, 1])` retournera `[2, 4]`.

Les **contraintes** à respecter pour chacune de vos deux fonctions sont les suivantes :

- vos fonctions doivent être **exclusivement récursives**, c'est-à-dire qu'il vous est interdit d'utiliser une boucle, que ce soit une boucle `while` ou une boucle `for` ;
- à l'exception de la fonction `len`, **vous ne pouvez pas utiliser de fonctions ou de méthodes existantes en Python**. Il vous est donc interdit, par exemple, d'utiliser la fonction `filter` ou la méthode `extend` des listes ;
- par contre, vous êtes autorisés à utiliser les **opérateurs** sur les listes comme `+`, `[:]`, `*`, etc.
- aucune de vos deux fonctions ne peut avoir d'effet de bord.

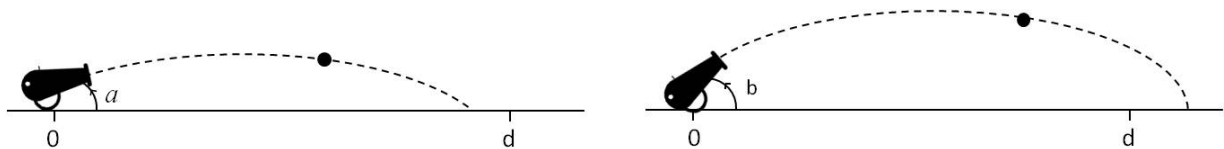
Problème 4 : le canonnier

Le but de cet exercice est de trouver un angle α (compris entre 0 et 45 degrés, par rapport à l'horizontale) d'un canon permettant de toucher une cible située à une distance d , au moyen de la méthode décrite ci-après.



Cette méthode utilise deux angles a et b ($\in [0, 45]$) représentant l'intervalle $[a, b]$ dans lequel on recherche à estimer la valeur de α . Ces angles sont choisis de telle sorte que le tir soit trop court avec l'angle a et trop long avec l'angle b (*précondition*). Ces préconditions impliquent donc que $a \leq b$. A chaque étape de cet algorithme, il faut calculer la moyenne c des angles a et b . Ensuite,

- si le boulet tombe à un point suffisamment proche de la cible, avec l'angle c , pour la toucher (notez que le boulet a un rayon r), nous avons trouvé un bon angle ;
- sinon, on passe à l'étape suivante avec les deux angles a et c ou b et c vérifiant les préconditions précitées.



Si nous négligeons la poussée d'Archimède et les frottements dus à l'air, les lois de Newtons nous permettent d'obtenir la distance d_{pc} à laquelle se trouve le point de chute du boulet en fonction de la vitesse de propulsion v et l'angle α du canon par rapport à l'horizontale :

$$d_{pc} = \frac{v^2 \times \sin(2\alpha)}{9,81}$$

Question :

1. Donnez le code python d'une fonction `dpc(v, alpha)` qui étant donné une vitesse de propulsion v et un angle $\alpha \in [0, 45]$, retourne la distance à laquelle se trouve le point de chute du boulet.

Remarque : La fonction `sin` du module `math` prend en paramètre un angle exprimé en radians. Ce module fourni également une fonction `radians(a)` qui prend en paramètre un angle a exprimé en degrés et retourne ce même angle exprimé en radians.

2. Donnez le code python d'une fonction purement **iterative** (sans aucun appel récursif) `angle(d, a, b, v, r)` qui étant donnés une distance d , deux angles a et b ($\in [0, 45]$), une vitesse de propulsion v et un rayon r du boulet, calcule un angle permettant de toucher la cible située à une distance d , suivant la méthode décrite précédemment.